

PROG7311 POE PART 1

Student Number:	ST10435542
Programme Code:	BCAD0701 (3)
Module Lecturer:	Reece Wanvig
Module Code:	PROG7311
Date of Submission:	27-03-2026

I hereby declare that I did not plagiarise the content of this assignment and that this is my own work.

Assignment submitted via Turnitin: ☒ (Tick the Box)

Table of Contents

1	ARCHITECTURAL FRAMEWORK STRATEGY	1
1.1	Executive Summary & Introduction	1
1.2	Enterprise Architecture Framework: Zachman Framework	1
1.2.1	Zachman Framework Matrix	2
2	DESIGN PATTERN SELECTION	4
2.1	Gang of Four (GoF) Design Patters:	4
2.1.1	The Abstract Factory Patter (Creational)	4
2.1.2	The Strategy Pattern (Behavioural)	4
2.1.3	The State Pattern (Behavioural)	5
3	NON-FUNCTIONAL REQUIREMENTS & SCALABILITY	7
3.1	Availability	7
3.2	Interoperability	7
3.3	Scalability Strategy	7
3.3.1	Horizontal Scaling	8
3.3.2	Load Balancing	8
3.3.3	Caching	8
4	REFERNCE LIST	9

Table of Figures

Figure 1 - UML Diagram (Designed on Draw.io)	6
--	---

1 ARCHITECTURAL FRAMEWORK STRATEGY

1.1 Executive Summary & Introduction

TechMove Logistics currently operates on severely outdated legacy system which comprises of spreadsheets, emails and manual phone calls. This fragmented system comes with massive risks for the company which resulted in lost invoices, compliance failures due expired contracts, and delayed processing.

In order for TechMove to remain competitive and be able to handle future growth, they will require a much stronger, up-to-standard system, which is what will be proposed in the Global Logistics Management System (GLMS). The proposal for an improved system will modernise TechMove's operations by moving away from their current monolithic, manual workflow to a scalable, Service-Oriented Architecture (SOA) hosted within a cloud-native, containerised environment (Docker).

1.2 Enterprise Architecture Framework: Zachman Framework

The Zachman Framework is an enterprise framework that describes a business project from different perspectives typically through a 6 by 6 table that consists of rows and columns, each representing a different view (The Independent Institute of Education, 2026). For the rows, they consist of the audiences, such as the owner, planner and designer, whereas the columns fill in those gaps by answering the fundamentals questions of What (the information), Where (the network stored), How (how it functions), Who (the people involved), When (timeline), and Why (the motivation to support the need) (Visual Paradigm, 2026).

Zachman is the ideal choice for TechMove because of its capability of categorizing all the specific technical and business details into one structured matrix, whereas TOGAF focuses more the sequential process of things, which in this case is not what we need (Raza, 2020). What we need is a structured overview, which is why I approached it using the Zachman framework.

TechMove's requirements are very specific in regards to their How's, such as their envisioned functional rules of converting currencies and PDF validation, as well as the

exact entities to include, and a specific breakdown of the network infrastructure (Docker containers).

With that being said, Zachman allows the freedom to map our top priorities for input under planner row (Board of Directors) against the software architecture, which would fall under the designer row, which ensures that all requirements are met (Visual Paradigm, 2026).

1.2.1 Zachman Framework Matrix

Zachman Perspective	What (Data)	How (Function)	Where (Network)
Planner	Logistics entities (Clients, Contracts, Driver Schedules, Invoices)	- Employ faster methods to avoid workflow bottlenecks - Automate currency conversions - Prevent compliance failures (expired contracts and business processing)	GLMS (able to withstand high growth and capacity by exhausting multiple resources to balance the systems load and prevent system failure)
Designer	SQL Database entities: Client - Name, Region Contract	- Backend APIs to handle the currency conversions	SOA using Docker containers: - sql-server-db

	- StartDate, EndDate, Status ServiceRequest - Description, Cost	- File validation logic for uploading contracts - Contract status validation logic (ensuring contracts aren't expired)	- glms-backend-api - glms-frontend-web
--	--	---	---

(Visual Paradigm, 2026)

2 DESIGN PATTERN SELECTION

2.1 Gang of Four (GoF) Design Patterns:

2.1.1 The Abstract Factory Patter (Creational)

The Factory pattern does a good job at centralising object creation, which is what TechMove requires in order to process the complexity of logic between its clients, contracts and service requests (Source Making, 2026). This ensures that the MVC, which acts as the presentation layer (translates the data), is completely separated from the service layer (gathers the data and disperses it accordingly), which is important for the transition to SOA (Source Making, 2026). Modular development is crucial for TechMove to ensure it is scalable and can withstand future growth.

We will create an IContractFactory interface which will be used by the LogisticsContractFactory class through Dependency Injection, and will handle all the complex instantiation of Contracts objects (Source Making, 2026).

2.1.2 The Strategy Pattern (Behavioural)

The GLMS should be able to automatically handles currency conversions using external APIs, and because exchange rate calculations vary from time to time, the Strategy pattern will allow us to choose the specific currency conversion algorithm at runtime without modifying the code that communicates with the APIs, making the core logic independent from the clients (APIs) that use it (Source Making, 2026).

We will design an ICurrencyConversionStrategy interface that contains a “convert” method that implements classes like UsdToZarStrategy or EurToZarStrategy through an external API (HttpClient). The ServiceRequest controller will call one of the APIs to execute the injected strategy (currency conversion), which will ultimately keep the system open for extension but closes for modification (can add more conversion strategies without the need to change core logic in the controller) (Source Making, 2026).

2.1.3 The State Pattern (Behavioural)

The system requirements made clear the need for strict workflow rules such as a service request cannot be made if a linked entity (Contract), which in this case is the parent entity, is expired or on hold. With that being said, using the State pattern encases the logic to validate this process inside a state-specific classes (ExpiredState/OnHoldState, as seen in *figure 1*), which eliminates the need for if/else statements in the business logic (tends to be hard to maintain) (Source Making, 2026).

To successfully follow through with this design pattern, we will create an IContractState interface with a supporting method (CanRaiseServiceRequest()), that will be linked to various “state” classes. These classes will implement the interface in a way that when a user attempts to raise a request, the Contract object will communicate with the validation logic and return the contracts state to its respective class, returning either true or false, determining the possibility of raising a request or not (Source Making, 2026).

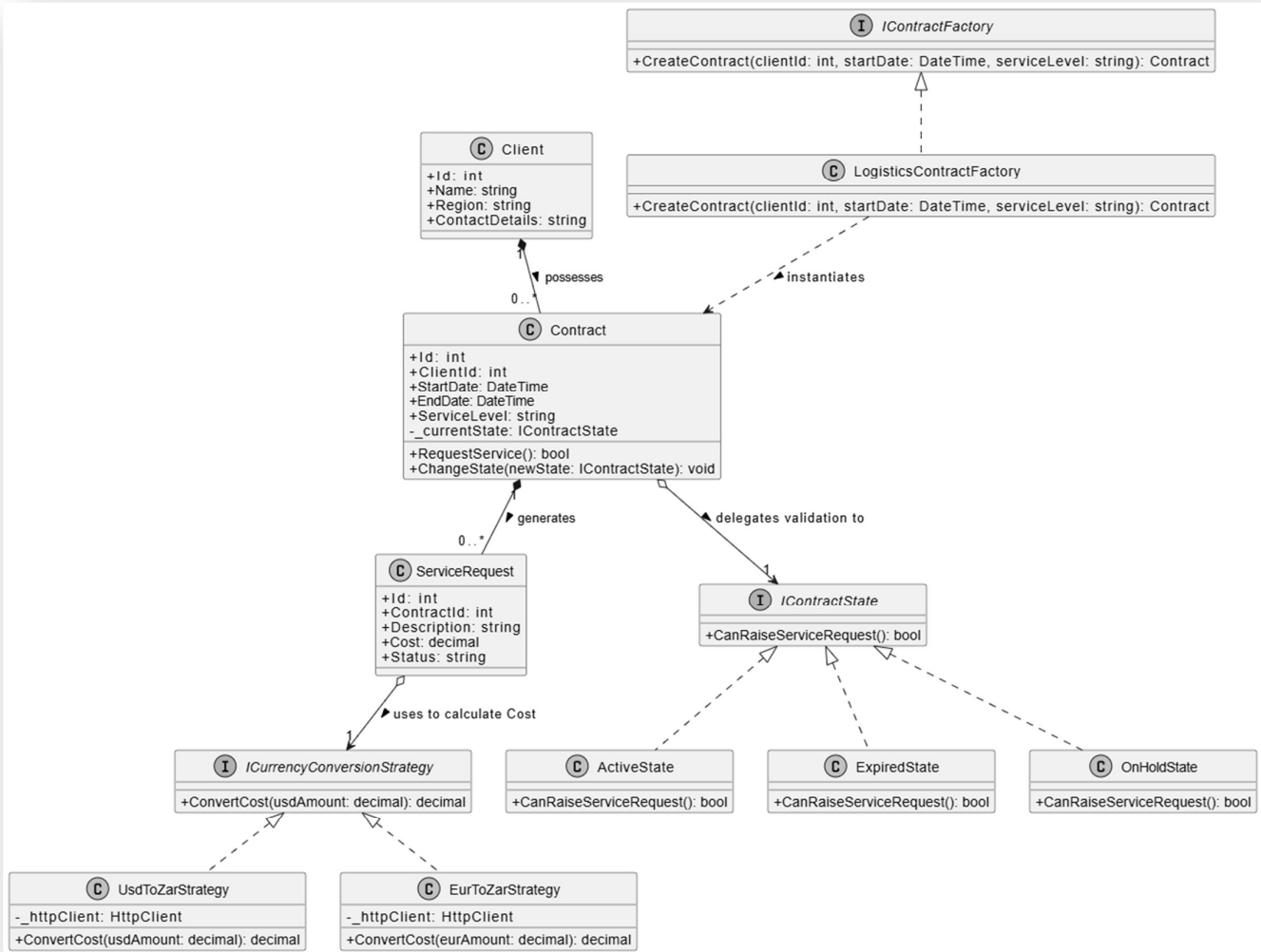


Figure 1 - UML Diagram (Designed on Draw.io)

3 NON-FUNCTIONAL REQUIREMENTS & SCALABILITY

3.1 Availability

Availability ensures that a system remains operational and accessible despite environmental impact (power/system failure) (Nourhansaeed, 2024). In order for the GLMS to achieve high availability, the use of Docker containers is essential (The Independent Institute of Education, 2026). The modular design of containers allows us to configure failovers and restart policies over the various individual containers for backend API, frontend and database, setting up a safety net if an error should ever occur in the future, the system will always recover automatically (The Independent Institute of Education, 2026). Hypothetically, if the “glms-frontend-web” container fails, Docker will automatically spin up a replacement without crashing the entire system, all due to the rules (restart policies/failovers) we configured upon creating the containers.

3.2 Interoperability

Data exchange is an important concept in the business world, especially for it to be a seamless, smooth process. Interoperability is the ability of a system (GLMS) to exchange data seamlessly with external systems (The Independent Institute of Education, 2026). For TechMove, this is quite an important factor for the financial part of things. The system will achieve Level 2 (Syntactic, which prioritizes the formatting of data) to Level 3 (Semantic, which practically decodes what the data says, creating an understand of what has been received) interoperability by using HTTP protocols and JSON data formats to communicate with the external currency exchange APIs (The Independent Institute of Education, 2026).

3.3 Scalability Strategy

If TechMove by chance lands a deal with a competitor overnight and is given 50,000 new contracts, this will obviously have an impact on the system with its dramatic spike in contract intake. Therefore, in order to handle a situation like this, it's important to implement these simple yet effective strategies.

3.3.1 Horizontal Scaling

The smarter move to handling system upgrades would be to scale horizontally by creating multiple identical Docker containers rather than upgrading the hardware of a single server (The Independent Institute of Education, 2026). This way the system is able to process and store more data at once, accommodating the increased workload (The Independent Institute of Education, 2026).

3.3.2 Load Balancing

Using the same example mentioned in 3.3.1, a load balancer helps direct and distribute all incoming traffic of contracts across the multiple containers (GeeksForGeeks, 2026). These load balancers are placed in front of each API instance to direct the “contract traffic” evenly across the available containers. This helps prevent any single API from being overwhelmed (GeeksForGeeks, 2026).

3.3.3 Caching

Memory is always the biggest challenge for large enterprises dealing with massive amounts of data. To reduce the immense load on a database, the implementation of caching for frequently read data (to save load times), such as the contract statuses and daily currency exchange rates (GeeksForGeeks, 2026). So, by storing this data in memory, it will significantly improve the systems response time, especially during periods with large influx (example explained under in 3.3.1) (GeeksForGeeks, 2026).

4 REFERENCE LIST

GeeksForGeeks, 2026. *Scalability in System Design*. [Online]
Available at: <https://www.geeksforgeeks.org/system-design/what-is-scalability/>
[Accessed 23 March 2026].

Nourhansaeed, 2024. *Functional VS Non-Functional Requirements in system design*. [Online]
Available at: <https://medium.com/@nourhansaeed707/functional-vs-non-functional-requirements-in-system-design-6b1b1a3785da>
[Accessed 23 March 2025].

Raza, M., 2020. *TOGAF vs Zachman: What's The Difference?*. [Online]
Available at: <https://www.bmc.com/blogs/togaf-vs-zachman/>
[Accessed 22 March 2026].

Source Making, 2026. *Abstract Factory Design Pattern*. [Online]
Available at: https://sourcemaking.com/design_patterns/abstract_factory
[Accessed 23 March 2026].

Source Making, 2026. *State Design Pattern*. [Online]
Available at: https://sourcemaking.com/design_patterns/state
[Accessed 22 March 2026].

Source Making, 2026. *Strategy Design Pattern*. [Online]
Available at: https://sourcemaking.com/design_patterns/strategy
[Accessed 23 March 2026].

The Independent Institute of Education, 2026. *PROGRAMMING 3A PROG7311 ENTERPRISE APPLICATION DEVELOPMENT EDPD7111 MODULE MANUAL* 2026. 1st ed. s.l.:The Independent Institute of Education (Pty) Ltd.

Visual Paradigm, 2026. *What is Zachman Framework?*. [Online]
Available at: <https://www.visual-paradigm.com/guide/enterprise-architecture/what-is-zachman-framework/>
[Accessed 24 March 2026].